UNIVERSITY OF PASSAU

6061 | Introduction to Deep Learning

Exercise 2

Prof. Dr. Florian Lemmerich[1] Andreas Einwiller[1] Max Klabunde[1*]

[1]Professorship for Applied Machine Learning
University of Passau

[*](2021–2025) Preparation, creation, and presentation

22.10.2026

Goal for this week

UNIVERSITY OF PASSAU

- You will be able to write a training script for a simple neural network
- ... and understand the basics of PyTorch

PyTorch

- Similar to `NumPy`: a library for array (tensor) operations
- ... with GPU support
- Automatic differentiation engine
- Library of neural network components
- Library of optimization tools

Graphics Processing Unit (GPU) support

GPU $\approx$ a processor with *many* cores

CPU	GPU
Several cores	Many cores (e.g., NVIDIA H100: 456 "Tensor Cores")
Low latency	High throughput
Good for serial processing	Good for parallel processing
Can do a handful of operations at once	Can do thousands of operations at once

Kevin Krewell (2009)

- Neural network (NN) operations are highly parallelizable
- GPUs can handle tens of thousands of concurrent threads
- $\rightarrow$ Tensor Cores accelerate large matrix operations

Detour: minimizing functions analytically

How do you find the minimum of $f(x) = x^2 - 3x$?

- Compute the derivative
- Find all x where $f'(x) = 0$
- Use the second derivative to check whether each critical point is a minimum or a maximum

What if, e.g.,

$$f(\boldsymbol{W}^{(1,2)}, \boldsymbol{b}^{(1,2)}, \boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{y}^T \tanh(\boldsymbol{W}^{(2)} \max(0, \boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}) + \boldsymbol{b}^{(2)})$$

Conclusion: non-linearity (due to activation functions)
$\rightarrow$ we need numerical methods

Why does it matter?

- Neural networks (NNs) are functions with many parameters
  - Input x is fixed; during a forward pass, the parameters are also fixed
  - A simple NN: $\hat{y}^{(pred)} = \tanh(\boldsymbol{W}^{(2)} \max(0, \boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}) + \boldsymbol{b}^{(2)})$
- Weights (W) and biases (b) are *trained to minimize a loss function*
- E.g., for a regression problem we might want

$$\min_{\boldsymbol{W}^{(1)},\boldsymbol{W}^{(2)},\boldsymbol{b}^{(1)},\boldsymbol{b}^{(2)}} \sum_{i=1}^{N} \left(\hat{y}_i^{(pred)} - y_i^{(true)} \right)^2$$

Minimizing functions numerically

- *Gradient Descent* is a general method to approximately minimize / optimize a function; α is the learning rate

$$w^{t+1} = w^t - \alpha \frac{\partial f}{\partial w}(w^t, \dots)$$

  Repeat until convergence or until the compute budget is exhausted
- We will discuss this in more detail in the coming weeks
- For now: we need derivatives (gradients) of the loss with respect to the parameters; not computed by hand! $\rightarrow$ PyTorch

PyTorch

- Similar to `NumPy`: a library for array (tensor) operations
- ... with GPU support
- **Automatic differentiation engine**
- Library of neural network components
- Library of optimization tools

Demo

PyTorch demo in a Jupyter notebook

Feedback

Please use the text field



https://lgms.nl/p/presentation-feedback/?code=d0ebb1

Time to do it yourself

UNIVERSITY OF PASSAU

- Work through the homework Jupyter notebook
- Reminder: `uv` setup + VS Code or Jupyter server
- If you missed last week's session, complete the installation steps first

Next week: gradient descent in more detail